

How the audio is built, from mic to ears

A walk-through of the audio path for someone who knows spatial audio well but has not read this code: what each stage does to the signal, in which convention, and where it lives.

The engine's method-by-method reference is `docs/core-api.md` in the repository. This page explains why the audio is wired the way it is.

■ Bed

The 4-channel first-order ambisonic recording made next to the camera (Rode NT-SF1). It is the whole concert as heard from there, with every musician and the room in it. It rotates with the head and is never split up.

■ Stems

One close-mic recording per musician. They are the "objects", the sources that get placed, raised and moved one by one. The same musicians are *also* inside the bed.

The two words come from object-based audio (bed + objects). Throughout this page, teal is the bed and amber is the stems.

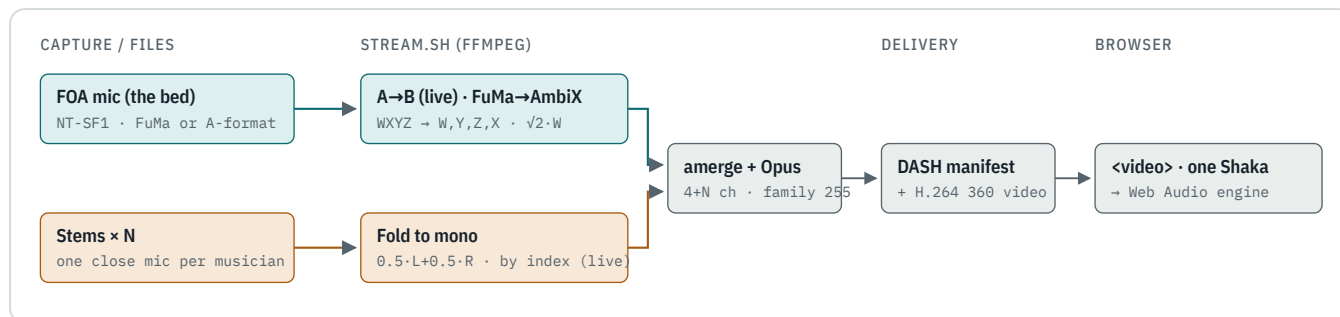
1. The picture in one paragraph

We transport **one Opus track** with `4 + N` discrete channels: a first-order ambisonic **bed** (AmbiX, ACN/SN3D) plus **one mono stem per musician**. It travels inside the same DASH manifest as the 360 video, so audio and video share one timeline.

In the browser, a single `<video>` element plays both, and its audio is taken into Web Audio with `createMediaElementSource`. There the bed is rendered binaurally with **Omnitone** (head-tracked FOA → HRTF), and each stem goes through its own **PannerNode** (HRTF) placed where the musician is. The stems are silent by default. They come up when the listener **looks at a musician and zooms** (VR and desktop), or they are always on and positioned in the real room (AR).

W	Y	Z	X	DR	GT	KEY	SAX	TBONE	TPT
0	1	2	3	4	5	6	7	8	9

The track of the current test take: the bed in AmbiX order, then one channel per musician, in the order of `stems[]`.



Both lanes meet in a single audio track, so a stem and its image in the bed are sample-aligned.

2. The data that drives it: `scene.json`

The editor (`editor.html`) writes `scene.json` , and both the encoder (`stream.sh` , through `server.js`) and the player read it. These are the audio fields:

FIELD	MEANING	USED BY
<code>bed</code>	4-channel WAV, the FOA bed	encoder
<code>bedFormat</code>	<code>fuma</code> (W,X,Y,Z, W at −3 dB) or <code>ambix</code>	encoder
<code>stems[i].file</code>	the musician's recording (folded to mono)	encoder
<code>stems[i].azimuthDeg / elevationDeg</code>	where the musician is in the 360 image	player (VR, desktop)
<code>stems[i].gainDb</code>	per-take level trim (the stems are not level-matched)	player
<code>stems[i].ar.{x,y,z}</code>	position in metres in the room	player (AR)
<code>alignment.{yawOffsetDeg, mirror}</code>	how the mic was rotated vs. the camera; rotates the bed only	player
<code>spotlight.{restGain, maxBoost, focusExp, zoomMax, bedDuck}</code>	the look + zoom behaviour	player
<code>sources.{distanceModel, refDistance, rolloffFactor, ...}</code>	<code>PannerNode</code> distance curve (matters in AR)	player
<code>ar.bedGain</code> , <code>ar.focus</code>	bed level and dwell-focus settings in AR	player

FIELD	MEANING	USED BY
<code>live.audio.*</code>	device, channel indices, A-format flag, A/V delay	encoder (live)

The order of `stems[]` is the channel order. Stem `i` is channel `4 + i` of the stream. Nothing else links a stem file to its channel.

3. Encoding: `stream.sh`

VOD (files)

1. **Bed.** It must have exactly 4 channels (`make-bed.sh` extracts them if an editor exported something like a 7.1 with duplicated channels). It is converted to AmbiX if needed: `FuMa [W,X,Y,Z] → AmbiX [√2·W, Y, Z, X]`.
2. **Stems.** Each file is folded to mono (`0.5·L + 0.5·R`).
3. **Merge.** `amerge` produces `4 + N` channels: `[W, Y, Z, X, stem0, ...]`.
4. **Encode.** `libopus -mapping_family 255` (discrete channels, no stereo coupling, no layout), 48 kHz, default bitrate `160k + 64k × N`.

Live (the mixer)

All audio arrives through **one** multichannel device (the mixer over ALSA, or one UDP stream). The FOA and each stem are **picked by channel index** (`live.audio.foa.ch` , `STEM_CH`).

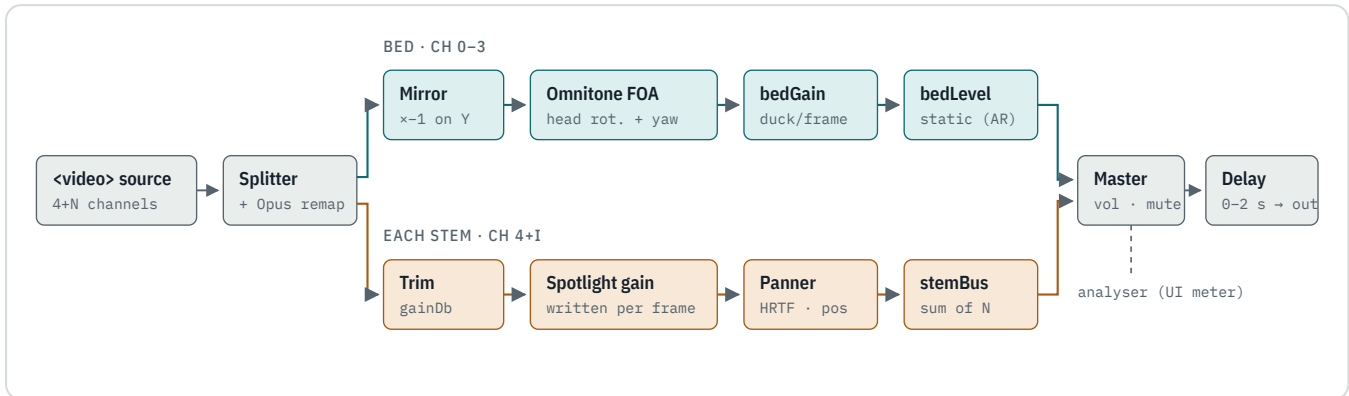
- If the mic sends **A-format** (NT-SF1 capsules), a fixed 4×4 A→B matrix turns it into FuMa, followed by the FuMa→AmbiX step above. This is a plain sum/difference matrix, **without the capsule equalisation** that the NT-SF1's own plugin applies.
- `adelay` delays **all** audio to match the video, which comes out later because of the camera's stitching. The delay depends on the setup (`live.audio.delay`).

Delivery

A single DASH manifest: H.264 video in MP4 segments, Opus audio in WebM segments (the Quest plays multichannel Opus only in WebM). A single Shaka instance plays it all in one `<video>`, so A/V sync comes from the player's construction, not from anything we correct.

4. Rendering: the Web Audio graph

The class is `ImmersiveAudioEngine` (`src/audio/ImmersiveAudioEngine.js`). Once `attach(videoEl)` has run, the graph is:



A second splitter also taps channels 0–3 into four analysers for `getDominantDirection()`, an intensity-vector direction-of-arrival estimate.

Stage by stage

- **Channel remap** (`opusChannelMap.js`). Chromium, and so the Quest browser, re-orders multichannel Opus to **Vorbis order** when a track has 3–8 channels, even with mapping family 255. ffmpeg does not do this, so a file that tests correctly offline can reach Web Audio shuffled. The engine undoes it with a source→output table. Only the 7-channel row (4 + 3 stems) has been measured on a Quest. The other rows come from Chromium's table. Above 8 channels there is no re-order. `channel-test.html` plays one tone per channel so you can check it.
- **Bed renderer.** By default this is `Omnitone.createFOARenderer`, fed with ACN/SN3D. Its HRIRs are embedded in the library, and the head rotation is applied inside it. `?renderer=hoast` switches to the older HOAST chain instead (rotator → matrix multiplier → decoder), which decodes to cardioids without real HRTFs. That chain is only a fallback.
- **Alignment.** This fixes the mic-vs-camera rotation of the take. `yawOffsetDeg` pre-rotates the head matrix passed to Omnitone, and `mirror` flips the sign of Y (ACN 1). Both apply to **the bed only**. The stems are placed by the video, so they are already aligned with it.
- **Stem chain.** A mono channel goes through the trim (the take's `gainDb`), then the spotlight gain (written every frame), then a `PannerNode` with `panningModel = 'HRTF'`. That is the **browser's built-in HRTF set, not Omnitone's**. The Web Audio `AudioListener` gets the same head pose as Omnitone.
- **Master.** It holds the volume and mute, then a `DelayNode` that can only *delay* audio (`?audiodelay=ms`, used when the audio is still early). The analyser sits before the delay and only feeds the UI meter.

5. Where the stems are: two geometries

This is where most of the confusion comes from. A stem has one of two geometries:

	NOT ANCHORED (360: DESKTOP, VR)	ANCHORED (AR PASSTHROUGH)
Position set by	<code>azimuthDeg</code> / <code>elevationDeg</code>	<code>ar.{x,y,z}</code> in room metres, via an <code>Object3D</code> (<code>bindStemToObject</code>)
Lives on	a sphere of radius 1 that moves with the head , like the 360 video sphere	fixed in the room
Walking closer	changes nothing (the sphere follows you)	changes direction and distance: the <code>PannerNode</code> distance model applies
Direction used for "looking at"	the stem's own direction	stem position minus head position

Why a head-centred sphere: in VR the head sits about 1.6 m above the `local-floor` origin. If the stems were fixed in the world at radius 1, they would end up below and behind the listener. So on every head update the engine re-seats them at `head + dir`.

6. Behaviour per mode

	DESKTOP 360	VR (QUEST)	AR (PASSTHROUGH)
Head rotation from	camera matrix (mouse)	XR pose matrix	XR pose matrix (with position)
Zoom	camera FOV 100°→30° ⇒ factor 1→2.5	right joystick ⇒ normalised 0–1	none (0)
Stems at rest	silent (<code>restGain</code> 0)	silent	always on (<code>restGain</code> 1)
Stem comes up by	look + zoom (spotlight)	look + zoom (spotlight)	walking closer (distance) + dwell focus (+6 dB by default)
Bed	full, ducked by <code>bedDuck</code> when a stem is focused	same	lowered to <code>ar.bedGain</code> (0.35); the real room is the backdrop

DESKTOP 360		VR (QUEST)	AR (PASSTHROUGH)
360 video	decoded	decoded	not decoded (the <code><video></code> still carries the audio)

The spotlight, in formulas

From `_updateSpotlight()`, for every stem, once per frame:

```

aim      = max(0, dot(lookForward, dirToStem))    // 1 = straight ahead
w        = aim^focusExp * zN                     // zN = zoom normalised 0..1
gain     = (restGain + w * maxBoost)
           * (focused ? focusBoost : someoneElseFocused ? focusDuck : 1)
bedGain  = 1 - bedDuck * max_i(w_i)               // max forced to 1 while a
focus is declared

```

Every gain moves with `setTargetAtTime(..., 0.08 s)`.

The bed **still contains every musician**. The spotlight adds a copy of the stem on top of its own image in the bed, and `bedDuck` pulls the whole bed down (in every direction) to make room.

`zoomMin` and `zoomMax` are clamped to the range of the zoom-matrix table (1–2.5). A `zoomMax` of 3.6 in the scene gets clamped to 2.5, with a console warning.

7. Conventions: the traps

- **Axes.** Three.js, WebXR and Web Audio use $x = \text{right}$, $y = \text{up}$, $-z = \text{front}$. Ambisonics uses $x = \text{front}$, $y = \text{left}$, $z = \text{up}$. `ambisonicAxes.js` does the remap.
- **Azimuth in `scene.json`:** 0 is front and positive is to the **left** (the ambisonic convention), so `dir = (-sin az, 0, -cos az)` in Three.js space. The 360 viewer's yaw is `-azimuthDeg`.
- **FuMa vs AmbiX.** The player only accepts AmbiX. The $\sqrt{2}$ on W and the re-order happen **once**, in `stream.sh`. If a bed is already AmbiX, declaring it FuMa scrambles it without any audible error.
- **Channel re-order** in the browser (§4). If the "X" channel carries a saxophone, suspect the remap before anything else.
- **Level trims are properties of the take** (`gainDb`), not of the player. They are needed because every stem was recorded on a different mic and preamp.

8. How to listen and measure

WANT TO...	USE
Check a bed's channel order and measure the mic rotation, without listening	editor → <i>Analyse bed</i> , i.e. <code>POST /api/bed/analyze</code> (<code>tools/bedAnalysis.js</code> : intensity vector in the 300–1500 Hz band, over the frames where each stem dominates)
List a WAV's channels and levels, spot duplicated channels	<code>./make-bed.sh -l file.wav</code>
Check the browser's Opus channel order	<code>channel-test.html</code> (one tone per channel)
Try another renderer	<code>?renderer=hoast</code>
Try an alignment without editing the scene	<code>?yaw=180&amirror=1</code>
Override the stem azimuths	<code>?stems=-50,-30:5,...</code> (<code>az[:e1]</code>)
Disable the channel remap	<code>?chmap=identity</code>
Log the spotlight weights and gains	<code>?spotlog=1</code> , or <code>engine.getSpotlightState()</code> in the console
Hear whether head rotation reaches the renderer (VR)	<code>?audiotest=spin</code> (the sound field spins by itself)
Tune live from the console	<code>engine.setStemGainDb('SAX', 6)</code> , <code>setSpotlightParams({...})</code> , <code>setSourceParams({...})</code> , <code>setOutputDelay(ms)</code>
A/V drift diagnostics	<code>?avlog=1</code>

9. Known limitations: where help is welcome

These come from the current code. None of them is hidden, but none has been tackled by a spatial-audio specialist yet.

1 No acoustic zoom with the default renderer.

`setZoomByFactor()` only drives the stem spotlight under Omnitone. The HOAST zoom matrices (`zoom-matrix.js`) only run in the cardioid fallback. They are also 3rd-order (25×25) matrices, of which the FOA chain uses the top-left 4×4. That is a truncation, not a zoom designed for first order.

2 Two different HRTFs.

The bed uses Omnitone's HRIRs and the stems use the browser's `PannerNode` HRTF. The timbre and the externalisation change when a stem comes up.

3 The bed contains the musicians.

The ducking is broadband and omnidirectional, with no directional suppression of the focused source in the bed.

4 AR stems are dry.

They get only distance attenuation: no room, reflections or near-field cues.

5 Take calibration is manual.

Azimuths, `gainDb` and alignment are set by hand or by ear, and no loudness normalisation happens anywhere in the chain. Several musicians are still heard in a different place from where they are seen.

6 A-format conversion without the capsule EQ

in the live NT-SF1 path.

7 FOA only.

Moving to higher orders would affect the channel count (and the 3–8 channel re-order above), the bitrate and the renderer.

8 No perceptual evaluation

has been done yet. Telemetry already records the head pose and the focused musician per frame against media time (`docs/telemetry.md`). The participants' voice can be recorded alongside (`docs/voice-recording.md`).